

TUGAS 6
SISTEM OPRASI



Dosen : Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Darina Qothrun Nada [2410651047]

FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025-2026

BAB I ANALISIS DEADLOCK

Jalankan program *deadlock_simple.c* dan jelaskan:

1. Mengapa program mengalami deadlock?

Program Mengalami deadlock karena kedua thread saling menunggu resource yang sedang di pegang thread lain. Contohnya: Thread 1 mengunci Lock 1, lalu ingin mengambil Lock 2 (yang sedang dipegang Thread 2). Thread 2 mengunci Lock 2, lalu ingin mengambil Lock 1 (yang sedang dipegang Thread 1). Jadi keduanya akan menunggu satu sama lain tanpa pernah melepas lock sehingga program akan berhenti total.

2. Kondisi deadlock apa saja yang terpenuhi?

- Mutual Exclusion

Setiap lock hanya bisa dipakai 1 thread dalam 1 waktu

- Hold and Wait

Thread 1 memegang lock 1 sambil menunggu lock 2 dan thread 2 melakukan sebaliknya.

- No Preemption

Lock tidak bisa direbut paksa jadi hanya bisa dilepas oleh thread yang memasangnya.

- Circular Wait

Thread 1 menunggu resource milik thread 2, dan thread 2 menunggu resource milik thread 1, sehingga berbentuk lingkaran.

3. Gambar diagram resource allocation graph-nya

Thread 1 → Lock 1 → Thread 2 → Lock 2 → Thread 1 (Terus Berulang) atau dalam bentuk hubungan:

- T1 memegang Lock1
- T1 menunggu Lock2
- T2 memegang Lock2
- T2 menunggu Lock1

Ini membentuk cycle, yang merupakan indikator deadlock.

BAB II IMPLEMENTASI PENCEGAHAN

Modifikasi program *deadlock_simple.c* menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

2.1 Lock ordering

```
GNU nano 7.2      prevention_ordering.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Mutex Global
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi yang dijalankan oleh Thread (mengikuti Lock Ordering: selalu Lock 1 -> Lock 2)
void *thread_func(void *arg) {
    long thread_id = (long)arg;

    printf("Thread %ld: Mencoba mengambil Lock 1...\n", thread_id);
    // Langkah 1: Ambil Lock 1 (Semua thread harus melakukan ini pertama)
    pthread_mutex_lock(&lock1);
    printf("Thread %ld: Berhasil dapat Lock 1!\n", thread_id);

    sleep(1); // Simulasikan pekerjaan atau delay saat memegang kunci

    printf("Thread %ld: Mencoba mengambil Lock 2...\n", thread_id);
    // Langkah 2: Ambil Lock 2
    pthread_mutex_lock(&lock2);
    printf("Thread %ld: Berhasil dapat Lock 2!\n", thread_id);

    // Kunci berhasil didapatkan semua
    printf("Thread %ld: Semua kunci didapat. Melakukan pekerjaan kritis...\n", thread_id);
    sleep(2);

    // Lepaskan kunci
    pthread_mutex_unlock(&lock2);
    printf("Thread %ld: Melepaskan Lock 2.\n", thread_id);

    pthread_mutex_unlock(&lock1);
    printf("Thread %ld: Melepaskan Lock 1.\n", thread_id);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n");

    // Buat Thread 1 dan Thread 2
    pthread_create(&t1, NULL, thread_func, (void *)1);
    pthread_create(&t2, NULL, thread_func, (void *)2);

    // Tunggu kedua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("=== PROGRAM SELESAI TANPA DEADLOCK (Lock Ordering) ===\n");

    return 0;
}
```

Output :

```
darin@DARINAQNADA:~/praktikum6/deadlock_simple$ gcc prevention_ordering.c -o ordering -lpthread
darin@DARINAQNADA:~/praktikum6/deadlock_simple$ ./ordering
=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===
Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Mencoba mengambil Lock 2...
Thread 1: Berhasil dapat Lock 2!
Thread 1: Semua kunci didapat. Melakukan pekerjaan kritis...
Thread 1: Melepaskan Lock 2.
Thread 1: Melepaskan Lock 1.
Thread 2: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Semua kunci didapat. Melakukan pekerjaan kritis...
Thread 2: Melepaskan Lock 2.
Thread 2: Melepaskan Lock 1.
=== PROGRAM SELESAI TANPA DEADLOCK (Lock Ordering) ===
```

2.2 Try Lock dengan Retry

```
GNU nano 7.2                                prevention_trylock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h> // Untuk usleep()

// Mutex Global
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi untuk Thread 1 (Mengunci: 1, lalu 2)
void *thread1_func(void *arg) {
    long thread_id = (long)arg;
    int retry_count = 0;

    while (1) {
        printf("Thread %ld: Mencoba mengambil Lock 1...\n", thread_id);
        if (pthread_mutex_trylock(&lock1) == 0) { // Coba Lock 1
            printf("Thread %ld: Berhasil dapat Lock 1!\n", thread_id);

            // Coba ambil Lock 2
            printf("Thread %ld: Mencoba trylock Lock 2...\n", thread_id);
            if (pthread_mutex_trylock(&lock2) == 0) { // Coba Lock 2
                // BERHASIL DAPAT KEDUA KUNCI
                printf("Thread %ld: BERHASIL dapat Lock 1 dan Lock 2! (Retry: %d)\n", thread_id, retry_count);
                sleep(1);

                pthread_mutex_unlock(&lock2);
                pthread_mutex_unlock(&lock1);
                return NULL; // Thread selesai
            } else {
                // Gagal dapat Lock 2 (Lock 2 sedang dipegang Thread lain)
                printf("Thread %ld: Gagal trylock Lock 2. Melepaskan Lock 1 dan mencoba lagi.\n", thread_id);
                pthread_mutex_unlock(&lock1); // Mencegah deadlock dengan melepaskan kunci yang sudah dipegang

                // Jeda dan hitung retry
                retry_count++;
                usleep(100000); // Tunggu 100ms sebelum mencoba lagi
            }
        }
    }

    // Fungsi untuk Thread 2 (Mengunci: 2, lalu 1. Pola terbalik)
    void *thread2_func(void *arg) {
        long thread_id = (long)arg;
        int retry_count = 0;

        while (1) {
            printf("Thread %ld: Mencoba mengambil Lock 2...\n", thread_id);
            if (pthread_mutex_trylock(&lock2) == 0) { // Coba Lock 2
                printf("Thread %ld: Berhasil dapat Lock 2!\n", thread_id);

                // Coba ambil Lock 1
                printf("Thread %ld: Mencoba trylock Lock 1...\n", thread_id);
                if (pthread_mutex_trylock(&lock1) == 0) { // Coba Lock 1
                    // BERHASIL DAPAT KEDUA KUNCI
                    printf("Thread %ld: BERHASIL dapat Lock 2 dan Lock 1! (Retry: %d)\n", thread_id, retry_count);
                    sleep(1);

                    pthread_mutex_unlock(&lock1);
                    pthread_mutex_unlock(&lock2);
                    return NULL; // Thread selesai
                } else {
                    // Gagal dapat Lock 1, lepaskan Lock 2.
                    printf("Thread %ld: Gagal trylock Lock 1. Melepaskan Lock 2 dan mencoba lagi.\n", thread_id);
                    pthread_mutex_unlock(&lock2);

                    // Jeda dan hitung retry
                    retry_count++;
                    usleep(100000); // Tunggu 100ms sebelum mencoba lagi
                }
            }
        }
    }

    int main() {
        pthread_t t1, t2;

        printf("=== PENCEGAHAN DEADLOCK: TRY-LOCK & RETRY ===\n");

        pthread_create(&t1, NULL, thread1_func, (void *)1);
        pthread_create(&t2, NULL, thread2_func, (void *)2);

        pthread_join(t1, NULL);
        pthread_join(t2, NULL);

        printf("=== PROGRAM SELESAI TANPA DEADLOCK (Try-lock) ===\n");

        return 0;
    }
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark M-J To Bracket
Exit Read File Replace Paste Justify Go To Line M-E Redo M-G Copy ^Q Where Was

Output :

```
darin@DARINAQNADA:~/praktikum6/deadlock_simple$ gcc prevention_trylock.c -o trylock -lpthread
darin@DARINAQNADA:~/praktikum6/deadlock_simple$ ./trylock
=== PENCEGAHAN DEADLOCK: TRY-LOCK & RETRY ===
Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 1: Mencoba trylock Lock 2...
Thread 1: BERHASIL dapat Lock 1 dan Lock 2! (Retry: 0)
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Mencoba trylock Lock 1...
Thread 2: BERHASIL dapat Lock 2 dan Lock 1! (Retry: 10)
=== PROGRAM SELESAI TANPA DEADLOCK (Try-lock) ===
```

2.3 Perbandingan Lock Ordering

Kelebihannya adalah efisien, simple, dan tidak ada retry, sedangkan kekurangannya adalah butuh desain urutan lock yang konsisten di seluruh program.

Try_Lock dan Retry

Kelebihannya adalah fleksibel, tidak harus sama dengan urutan lock nya, sedangkan kekurangannya adalah bisa banyak loop retry jadi tidak efisien.

BAB III

BANKER'S ALGORITHM

Buat Program baru dengan data:

- Proses
- jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Nano bankers.c

```
GNU nano 7.2                                bankers.c *
#include <stdio.h>
#include <stdbool.h>

// Definisi Konstanta
#define NUM_PROCESSES 4 // P0, P1, P2, P3
#define NUM_RESOURCES 3 // Resource A, B, C

// --- DATA AWAL ---

// Kebutuhan Maksimum (Max)
int Max[NUM_PROCESSES][NUM_RESOURCES] = {
    {6, 0, 1}, // P0
    {1, 1, 3}, // P1
    {4, 2, 1}, // P2
    {2, 1, 1}  // P3
};

// Sumber Daya yang dialokasikan saat ini (Allocation)
int Allocation[NUM_PROCESSES][NUM_RESOURCES] = {
    {1, 0, 0}, // P0
    {1, 1, 0}, // P1
    {2, 0, 1}, // P2
    {0, 0, 0}  // P3
};

// Sumber Daya Tersedia (Available): [5, 4, 3]
int Available[NUM_RESOURCES] = {5, 4, 3};

int Need[NUM_PROCESSES][NUM_RESOURCES];
int safe_sequence[NUM_PROCESSES];

// --- FUNGSI ALGORITMA ---

// Fungsi untuk menghitung Matriks Kebutuhan (Need = Max - Allocation)
void calculate_need() {
    for (int i = 0; i < NUM_PROCESSES; i++) {
        for (int j = 0; j < NUM_RESOURCES; j++) {
            Need[i][j] = Max[i][j] - Allocation[i][j];
        }
    }
}

// Fungsi Safety Algorithm: Mengecek apakah state saat ini aman
bool is_safe_state(void) {
    int Work[NUM_RESOURCES];
    for (int i = 0; i < NUM_RESOURCES; i++) {
        Work[i] = Available[i];
    }

    bool Finish[NUM_PROCESSES] = {false};
    int count = 0;
    int k = 0;

    for(int i=0; i < NUM_PROCESSES; i++) safe_sequence[i] = -1;

    while (count < NUM_PROCESSES) {
        bool found = false;
        for (int p = 0; p < NUM_PROCESSES; p++) {
            if (Finish[p] == false) {
                int j;
                for (j = 0; j < NUM_RESOURCES; j++) {
                    if (Need[p][j] > Work[j]) {
                        break;
                    }
                }
                if (j == NUM_RESOURCES) {
                    // Found a safe process
                    for (int j = 0; j < NUM_RESOURCES; j++) {
                        Work[j] += Allocation[p][j];
                    }
                    Finish[p] = true;
                    count++;
                    k++;
                    safe_sequence[k] = p;
                }
            }
        }
    }
}
```

```

    }
}

if (j == NUM_RESOURCES) {
    // Sumber Daya dikembalikan ke Work
    printf(" -> P%d dapat dieksekusi (Work: A=%d, B=%d, C=%d)\n", p, Work[0], Work[1], Work[2]);
    for (int r = 0; r < NUM_RESOURCES; r++) {
        Work[r] += Allocation[p][r];
    }
    safe_sequence[k++] = p;
    Finish[p] = true;
    found = true;
    count++;
}
}

if (found == false && count < NUM_PROCESSES) {
    return false; // Tidak Aman
}

return true; // Aman
}

// Fungsi Resource Request Algorithm: Memproses permintaan
void request_resources(int process_id, int request[]) {
    int i = process_id;
    // Variabel temporer untuk rollback jika unsafe
    int temp_Available[NUM_RESOURCES];
    int temp_Allocation[NUM_PROCESSES][NUM_RESOURCES];
    int temp_Need[NUM_PROCESSES][NUM_RESOURCES];

```

```

    printf("\n--- Proses Permintaan Sumber Daya P%d ---\n", i);
    printf("Request: A=%d, B=%d, C=%d\n", request[0], request[1], request[2]);

    // 1. Cek Request <= Need
    for (int j = 0; j < NUM_RESOURCES; j++) {
        if (request[j] > Need[i][j]) {
            printf("⚠ DITOLAK: Permintaan melebihi kebutuhan maksimum (Need)!\n");
            return;
        }
    }

    // 2. Cek Request <= Available
    for (int j = 0; j < NUM_RESOURCES; j++) {
        if (request[j] > Available[j]) {
            printf("⚠ MENUNGGU: Permintaan melebihi Sumber Daya Tersedia (Available).\n");
            return;
        }
    }

    // --- Cek Keamanan Sementara (Trial Allocation) ---
    // Simpan status lama
    for (int j=0; j<NUM_RESOURCES; j++) temp_Available[j] = Available[j];
    for (int j=0; j<NUM_PROCESSES; j++) {
        for (int r=0; r<NUM_RESOURCES; r++) {
            temp_Allocation[j][r] = Allocation[j][r];
            temp_Need[j][r] = Need[j][r];
        }
    }

    // 3. Alokasi Sementara (Simulasi)

```

```

    // 3. Alokasi Sementara (Simulasi)
    for (int j = 0; j < NUM_RESOURCES; j++) {
        Available[j] -= request[j];
        Allocation[i][j] += request[j];
        Need[i][j] -= request[j];
    }

    // 4. Cek apakah state baru aman
    if (is_safe_state()) {
        printf("✅ Permintaan DITERIMA! State baru AMAN.\n");
        printf("Available Baru: A=%d, B=%d, C=%d\n", Available[0], Available[1], Available[2]);
        printf("Urutan Aman: <");
        for (int p = 0; p < NUM_PROCESSES; p++) {
            printf("P%d%s", safe_sequence[p], (p < NUM_PROCESSES - 1 ? " " : ""));
        }
        printf(">\n");
    } else {
        printf("❌ Permintaan DITOLAK! State baru TIDAK AMAN.\n");

        // Rollback (Mengembalikan state lama)
        for (int j = 0; j < NUM_RESOURCES; j++) {
            Available[j] = temp_Available[j];
        }
        for (int j=0; j<NUM_PROCESSES; j++) {
            for (int r=0; r<NUM_RESOURCES; r++) {
                Allocation[j][r] = temp_Allocation[j][r];
                Need[j][r] = temp_Need[j][r];
            }
        }
    }
}

```

```

// --- FUNGSI MAIN ---
int main() {
    calculate_need();

    printf("=== BANKER'S ALGORITHM (4 PROSES, 3 RESOURCE) ===\n");
    printf("Available Awal: A=%d, B=%d, C=%d\n", Available[0], Available[1], Available[2]);

    // Cek Keamanan State Awal
    printf("\n--- Cek Keamanan State Awal ---\n");
    if (is_safe_state()) {
        printf("Sistem Awal berada dalam KONDISI AMAN.\n");
    } else {
        printf("Sistem Awal berada dalam KONDISI TIDAK AMAN.\n");
    }

    // Simulasi Permintaan 1: P1 meminta A=0, B=0, C=3 (Aman)
    int request1[] = {0, 0, 3};
    request_resources(1, request1);

    // Simulasi Permintaan 2: P3 meminta A=1, B=0, C=0 (Tidak Aman / Ditolak)
    int request2[] = {1, 0, 0};
    request_resources(3, request2);

    return 0;
}

```

Penjelasan :

- $need[i][j] = max[i][j] - allocation[i][j];$
ini untuk menghitung kebutuhan sisa tiap proses.
- $work[j] = available[j];$
ini untuk menyalin resource yang tersedia untuk simulasi pengecekan keamanan.
- $if (need[i][j] > work[j])$
ini untuk mengecek apakah proses bisa dijalankan dengan resource yang ada
- $work[j] += allocation[i][j];$
ini jika resource telah selesai maka resource tersebut akan dikembalikan ke sistem
- Pretend Allocation (alokasi sementara request)
Sistem seolah seolah memberikan resource untuk mengecek apakah aman atau tidak.
- Rollback jika unsafe
Jika sistem tidak aman ,semua perubahan yang dilakukan akan dibatalakan untuk mencegah terjadinya deadlock
- $if (isSafe())$
ini adalah request yang hanya terjadi jika sistem tetap aman.

Output :

```
darin@DARINAQNADA:~/tugas6/brankers$ gcc bankers.c -o bankers
darin@DARINAQNADA:~/tugas6/brankers$ ./bankers
=== BANKER'S ALGORITHM (4 PROSES, 3 RESOURCE) ===
Available Awal: A=5, B=4, C=3

--- Cek Keamanan State Awal ---
-> P0 dapat dieksekusi (Work: A=5, B=4, C=3)
-> P1 dapat dieksekusi (Work: A=6, B=4, C=3)
-> P2 dapat dieksekusi (Work: A=7, B=5, C=3)
-> P3 dapat dieksekusi (Work: A=9, B=5, C=4)
Sistem Awal berada dalam KONDISI AMAN.

--- Proses Permintaan Sumber Daya P1 ---
Request: A=0, B=0, C=3
-> P1 dapat dieksekusi (Work: A=5, B=4, C=0)
-> P2 dapat dieksekusi (Work: A=6, B=5, C=3)
-> P3 dapat dieksekusi (Work: A=8, B=5, C=4)
-> P0 dapat dieksekusi (Work: A=8, B=5, C=4)
✅ Permintaan DITERIMA! State baru AMAN.
Available Baru: A=5, B=4, C=0
Urutan Aman: <P1, P2, P3, P0>

--- Proses Permintaan Sumber Daya P3 ---
Request: A=1, B=0, C=0
-> P1 dapat dieksekusi (Work: A=4, B=4, C=0)
-> P2 dapat dieksekusi (Work: A=5, B=5, C=3)
-> P3 dapat dieksekusi (Work: A=7, B=5, C=4)
-> P0 dapat dieksekusi (Work: A=8, B=5, C=4)
✅ Permintaan DITERIMA! State baru AMAN.
Available Baru: A=4, B=4, C=0
Urutan Aman: <P1, P2, P3, P0>
```

Cara kerja program:

1. Need dihitung otomatis → Max – Allocation
2. User memasukkan:
 - Nomor proses (0–3)
 - Resource yang ingin di-request (3 angka)
3. Program:
 - Mengecek apakah request valid
 - Melakukan alokasi sementara
 - Mengecek apakah sistem tetap aman
 - Jika aman maka request disetujui
 - Jika tidak maka rollback dan request ditolak

BAB IV

REAL CASE

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

Buat file `transfer_deadlock.c`

```
GNU nano 7.2 transfer_deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

// Struktur Akun Bank
typedef struct {
    int id;
    long balance;
    pthread_mutex_t lock;
} Account;

// Inisialisasi Akun Global
Account accountA = {101, 500, PTHREAD_MUTEX_INITIALIZER};
Account accountB = {102, 300, PTHREAD_MUTEX_INITIALIZER};

// Fungsi Transfer Dana yang Berpotensi Deadlock
void transfer_funds(Account *from, Account *to, long amount) {
    // 1. Thread mencoba mengunci akun "from"
    printf("Thread %lu: Mencoba mengunci Akun %d (dari)...\n", pthread_self(), from->id);
    pthread_mutex_lock(&from->lock);
    printf("Thread %lu: BERHASIL mengunci Akun %d.\n", pthread_self(), from->id);

    // Simulasi delay singkat agar thread lain bisa mengambil kunci lain
    usleep(100000); // 100ms

    // 2. Thread mencoba mengunci akun "to"
    // DEADLOCK terjadi di sini karena kedua thread saling menahan kunci dan menunggu kunci lain
    printf("Thread %lu: Mencoba mengunci Akun %d (ke)...\n", pthread_self(), to->id);
    pthread_mutex_lock(&to->lock);
    printf("Thread %lu: BERHASIL mengunci Akun %d.\n", pthread_self(), to->id);

    // Area Kritis: Melakukan transfer (Hanya akan dicapai jika TIDAK terjadi deadlock)
    if (from->balance >= amount) {
        from->balance -= amount;
        to->balance += amount;
        printf("Thread %lu: Transfer berhasil! %ld dari Akun %d ke Akun %d.\n",
            pthread_self(), amount, from->id, to->id);
    } else {
        printf("Thread %lu: Transfer GAGAL! Saldo Akun %d tidak mencukupi.\n", pthread_self(), from->id);
    }

    // Melepaskan kunci
    pthread_mutex_unlock(&to->lock);
    pthread_mutex_unlock(&from->lock);
}

// Fungsi yang dijalankan oleh Thread 1: A -> B
void *thread1_func(void *arg) {
    // Thread 1: Mencoba transfer 100 dari A ke B
    transfer_funds(&accountA, &accountB, 100);
    return NULL;
}

// Fungsi yang dijalankan oleh Thread 2: B -> A
void *thread2_func(void *arg) {
    // Thread 2: Mencoba transfer 50 dari B ke A
    transfer_funds(&accountB, &accountA, 50);
    return NULL;
}

int main() {
    pthread_t t1, t2;
}
```

```

printf("=== SIMULASI DEADLOCK TRANSFER DANA ===\n");
printf("Saldo Awal: Akun A = %ld, Akun B = %ld\n", accountA.balance, accountB.balance);

// Buat Thread 1 (A -> B) dan Thread 2 (B -> A)
pthread_create(&t1, NULL, thread1_func, NULL);
pthread_create(&t2, NULL, thread2_func, NULL);

// Tunggu kedua thread selesai (atau hang jika deadlock)
pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("\n=== PROSES SELESAI ===\n");
printf("Saldo Akhir: Akun A = %ld, Akun B = %ld\n", accountA.balance, accountB.balance);

// Perhatian: Jika deadlock terjadi, program akan hang setelah output sebagian!
return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-J To Bracket
 ^X Exit ^R Read File ^_ Replace ^V Paste ^G Justify ^/ Go To Line M-E Redo M-G Copy ^Q Where Was

Penjelasan :

- pthread_mutex_t rekening1, rekening2
ini untuk 2 mutex mewakili 2 rekening yang harus di lock sebelum transfer.
- pthread_mutex_lock(&rekening1) (A)
Ini untuk A mengunci rekening 1 terlebih dahulu.
- pthread_mutex_lock(&rekening2) (A)
ini untuk A mencoba mengunci rekening 2, tetapi akan gagal karena B sudah memegangnya.
- pthread_mutex_lock(&rekening2) (B)
ini adalah B mengunci rekening 2 terlebih dahulu.
- pthread_mutex_lock(&rekening1) (B)
ini untuk B mencoba mengunci rekening 1, tetapi gagal karena A sudah memegangnya.
- Hasil
A menunggu B dan B menunggu A (Deadlock)

Output :

```

darin@DARINAQNA:~/tugas6/transfer_deadlock$ gcc transfer_deadlock.c -o transfer_deadlock -lpthread
darin@DARINAQNA:~/tugas6/transfer_deadlock$ ./transfer_deadlock
=== SIMULASI DEADLOCK TRANSFER DANA ===
Saldo Awal: Akun A = 500, Akun B = 300
Thread 132220861806272: Mencoba mengunci Akun 101 (dari)...
Thread 132220861806272: BERHASIL mengunci Akun 101.
Thread 132220853413568: Mencoba mengunci Akun 102 (dari)...
Thread 132220853413568: BERHASIL mengunci Akun 102.
Thread 132220861806272: Mencoba mengunci Akun 102 (ke)...
Thread 132220853413568: Mencoba mengunci Akun 101 (ke)...
^C

```

Cara Kerja Program:

- A mengunci Rekening 1
- B mengunci Rekening 2
- A butuh Rek. 2 yang dikunci B
- B butuh Rek. 1 yang dikunci A

Jadi yang terjadi adalah kondisi terus saling menunggu dan terjadilah deadlock.

Buat file transfer_prevention_deadlock.c

```
GNU nano 7.2 transfer_prevention_deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

// Struktur Akun Bank
typedef struct {
    int id;
    long balance;
    pthread_mutex_t lock;
} Account;

// Inisialisasi Akun Global
Account accountA = {101, 500, PTHREAD_MUTEX_INITIALIZER};
Account accountB = {102, 300, PTHREAD_MUTEX_INITIALIZER};

// Fungsi Transfer Dana yang Mencegah Deadlock menggunakan Lock Ordering
void transfer_funds_safe(Account *from, Account *to, long amount) {
    // Tentukan kunci mana yang akan diambil pertama (Lock 1)
    pthread_mutex_t *lock1, *lock2;
    Account *first_account, *second_account;

    if (from->id < to->id) {
        // Urutan Normal: Kunci akun ID kecil (from) dulu, lalu ID besar (to)
        lock1 = &from->lock;
        lock2 = &to->lock;
        first_account = from;
        second_account = to;
    } else if (from->id > to->id) {
        // Urutan Terbalik: Kunci akun ID kecil (to) dulu, lalu ID besar (from)
        lock1 = &to->lock;
        lock2 = &from->lock;
        first_account = to;
        second_account = from;
    } else {
        // Transfer ke diri sendiri, tidak perlu kunci
        printf("Thread %lu: Transfer ke akun yang sama. Diabaikan.\n", pthread_self());
        return;
    }

    // 1. Ambil Kunci Pertama (ID kecil)
    printf("Thread %lu: Mencoba mengunci Akun ID %d (pertama)...\n", pthread_self(), first_account->id);
    pthread_mutex_lock(lock1);
    printf("Thread %lu: BERHASIL mengunci Akun ID %d.\n", pthread_self(), first_account->id);

    // 2. Ambil Kunci Kedua (ID besar)
    printf("Thread %lu: Mencoba mengunci Akun ID %d (kedua)...\n", pthread_self(), second_account->id);
    pthread_mutex_lock(lock2);
    printf("Thread %lu: BERHASIL mengunci Akun ID %d.\n", pthread_self(), second_account->id);

    // Area Kritis: Melakukan transfer
    if (from->balance >= amount) {
        from->balance -= amount;
        to->balance += amount;
        printf("Thread %lu: Transfer berhasil! %d dari Akun %d ke Akun %d.\n",
            pthread_self(), amount, from->id, to->id);
    } else {
        printf("Thread %lu: Transfer GAGAL! Saldo Akun %d tidak mencukupi.\n", pthread_self(), from->id);
    }

    // Melepaskan kunci dengan urutan terbalik
    pthread_mutex_unlock(lock2);
    pthread_mutex_unlock(lock1);
}

// Fungsi yang dijalankan oleh Thread 1: A (ID 101) -> B (ID 102)
void *thread1_func(void *arg) {
    // Urutan penguncian internal: Lock 101, lalu Lock 102
    transfer_funds_safe(&accountA, &accountB, 100);
    return NULL;
}

// Fungsi yang dijalankan oleh Thread 2: B (ID 102) -> A (ID 101)
void *thread2_func(void *arg) {
    // Urutan penguncian internal: Lock 101, lalu Lock 102 (Walaupun B yang memulai, ia harus mengunci A dulu)
    transfer_funds_safe(&accountB, &accountA, 50);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK TRANSFER DANA (LOCK ORDERING) ===\n");
    printf("Saldo Awal: Akun A (ID %d) = %ld, Akun B (ID %d) = %ld\n",
        accountA.id, accountA.balance, accountB.id, accountB.balance);

    // Buat Thread 1 (A -> B) dan Thread 2 (B -> A)
    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    // Tunggu kedua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== PROSES SELESAI ===\n");
    printf("Saldo Akhir: Akun A = %ld, Akun B = %ld\n", accountA.balance, accountB.balance);
}
```

```
printf("\n=== PROSES SELESAI ===\n");
printf("Saldo Akhir: Akun A = %ld, Akun B = %ld\n", accountA.balance, accountB.balance);

return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark M-] To Bracket
Exit Read File Replace Paste Justify Go To Line M-E Redo M-C Copy M-^ Where Was

Penjelasan:

- `lock_ordered(from, to)`
ini adalah fungsi menentukan urutan penguncian berdasarkan ID rekening.
- `first = from < to ? from : to`
ini untuk menentukan rekening mana yang lebih kecil ID nya.
- `pthread_mutex_lock(&rekening[first])`
ini agar selalu mengunci rekening nomor kecil lebih dahulu.
- `pthread_mutex_lock(&rekening[second])`
ini agar mengunci nomor besar.
- `unlock_ordered()`
ini untuk melepas lock dengan urutan terbalik.
- Hasil
Yang terjadi adalah semua thread mengunci rekening dengan urutan yang sama dan deadlock dicegah.

Output :

```
daring@DARINAQNADA:~/tugas6/transfer_prevention_deadlock$ nano transfer_prevention_deadlock.c
daring@DARINAQNADA:~/tugas6/transfer_prevention_deadlock$ gcc transfer_prevention_deadlock.c -o transfer_safe -lpthread
daring@DARINAQNADA:~/tugas6/transfer_prevention_deadlock$ ./transfer_safe
=== PENCEGAHAN DEADLOCK TRANSFER DANA (LOCK ORDERING) ===
Saldo Awal: Akun A (ID 101) = 500, Akun B (ID 102) = 300
Thread 128056729532096: Mencoba mengunci Akun ID 101 (pertama)...
Thread 128056729532096: BERHASIL mengunci Akun ID 101.
Thread 128056729532096: Mencoba mengunci Akun ID 102 (kedua)...
Thread 128056729532096: BERHASIL mengunci Akun ID 102.
Thread 128056729532096: Transfer berhasil! 100 dari Akun 101 ke Akun 102.
Thread 128056721139392: Mencoba mengunci Akun ID 101 (pertama)...
Thread 128056721139392: BERHASIL mengunci Akun ID 101.
Thread 128056721139392: Mencoba mengunci Akun ID 102 (kedua)...
Thread 128056721139392: BERHASIL mengunci Akun ID 102.
Thread 128056721139392: Transfer berhasil! 50 dari Akun 102 ke Akun 101.

=== PROSES SELESAI ===
Saldo Akhir: Akun A = 450, Akun B = 350
```

Cara kerja program:

- Setiap rekening harus dikunci sebelum dipakai, agar tidak dipakai 2 orang sekaligus
- Program selalu mengunci rekening dengan urutan yang sama, rekening bernomor kecil dulu, kemudian rekening bernomor besar
- Ketika A dan B mentransfer bersamaan:
A : Rekening 1 ke 2
B : Rekening 2 ke 1
Jadi keduanya akan tetap mengikuti aturan yang sama, yaitu kunci rekening 1 dulu lalu rekening 2
- Akibatnya

Hanya 1 orang yang bisa mendapatkan 2 kunci.yang lainnya otomatis menunggu sebentar tidak selamanya dan kemudian deadlock tidak akan terjadi.